Channel Duality as Role-Restricted Parallel: Embedding $S/\overline{S}$ into $S_A \parallel S_B$

Alexandre Zua Caldeira
Independent Researcher, Berlin, Germany
alex@zuacaldeira.com

March 2026

Abstract

In the π-calculus tradition, a communication channel has two endpoints typed by *dual* session types S and $\overline{S}$: what one side sends, the other receives. We show that this duality embeds cleanly into the *reticulate* framework, where session-type state spaces form bounded lattices. Concretely, a channel is an object with two roles: role A follows S and role B follows $\overline{S} = \text{dual}(S)$. The channel's state space is the product $\mathcal{L}(S) \times \mathcal{L}(\overline{S})$, which (1) is always a bounded lattice, (2) admits each role's local lattice as an order-embedding, and (3) exhibits *branch complementarity*—every branch transition for role A is a selection transition for role B, and vice versa. We further show that for binary global types G, projecting onto sender and receiver yields dual local types whose channel product matches the global state space. All results are verified mechanically across 71 benchmarks (plus 7 binary multiparty protocols) using the Reticulate tool.

Contents

1 Introduction **2**

2 Background **2**
2.1 Session Types . . . 3
2.2 State Spaces and the Reticulate Theorem . . . 3
2.3 Duality . . . 3
2.4 Product Construction . . . 3

3 The Channel Construction **4**

4 Branch Complementarity **4**
4.1 Complementarity in the Product . . . 5

5 Role Embedding **5**

6 Connection to Global Types **6**
6.1 From Channels to Multiparty . . . 6

7 Empirical Verification **7**
7.1 Setup . . . 7
7.2 Results . . . 7
7.3 Benchmark Highlights . . . 7

8 Related Work **8**

DRAFT

9 Conclusion 8

10 Frequently Asked Questions 8

1 Introduction

Session types [8] describe the communication protocols that govern interaction with objects or channels. In the binary setting introduced by Honda, Vasconcelos, and Kubo, a channel connects two endpoints: one typed by a session type S and the other by its *dual* $\overline{S}$. Duality ensures that sends match receives, branches match selections, and the protocol terminates consistently.

Gay and Hole [7] developed a subtyping theory for binary session types, where duality plays a central role: the subtyping relation reverses under duality, and type safety depends on the dual endpoints being compatible. Dardha, Giachino, and Sangiorgi [6] revisited the relationship between session types and linear logic, further connecting duality to logical negation.

In the *reticulate* programme [5], we have shown that the state space $\mathcal{L}(S)$ of any well-formed session type S forms a bounded lattice, and that duality preserves state-space structure up to isomorphism [4]. The present article takes the next step: we show that *a binary channel is naturally modelled as a parallel composition of dual local types*, and that the resulting product lattice has rich structural properties.

Contributions.

(i) We define the *channel construction*: given S, the channel state space is $\mathcal{L}(S) \times \mathcal{L}(\overline{S})$, and prove it is always a bounded lattice (Theorem 3.2).

(ii) We prove *branch complementarity*: branch transitions in $\mathcal{L}(S)$ correspond exactly to selection transitions in $\mathcal{L}(\overline{S})$ (Theorem 4.2).

(iii) We prove that each role embeds into the channel product via the *left/right inclusion* (Theorem 5.2).

(iv) We connect the channel construction to MPST projection: for binary global types, the channel product of the two projections is isomorphic to the global state space (Theorem 6.2).

(v) We verify all results across 71 benchmarks plus 7 binary multiparty protocols using Reticulate.

Organisation. Section 2 recalls the necessary background. Section 3 defines the channel construction and states the main theorems. Section 4 develops branch complementarity. Section 5 proves the role embedding theorem. Section 6 connects to global types. Section 7 reports the empirical verification. Section 8 discusses related work. Section 9 concludes. Section 10 provides a FAQ.

2 Background

We briefly recall the definitions and results from the reticulate programme that this article builds upon.

2.1 Session Types

A session type describes a protocol on an object or channel endpoint. The core grammar is:

$$S \ ::= \ \&\{m_1 : S_1, \ldots, m_n : S_n\} \ | \ \oplus\{l_1 : S_1, \ldots, l_n : S_n\} \ | \ S_1 \parallel S_2 \ | \ \mu X.\, S \ | \ X \ | \ \mathbf{end}$$

Branch (&) represents external choice: the environment chooses which method to call. Selection ($\oplus$) represents internal choice: the object decides which label to emit. Parallel ($\parallel$) models concurrent execution of two sub-protocols on a shared object. Recursion ($\mu X.S$) supports loops.

2.2 State Spaces and the Reticulate Theorem

The *state space* $\mathcal{L}(S)$ of a session type S is a labeled transition system (LTS) whose states represent protocol stages and whose transitions represent method calls or label emissions. It has a unique initial state $\top$ (top) and terminal state $\bot$ (bottom), with the reachability ordering $s_1 \sqsupseteq s_2$ iff there is a path from s_1 to s_2.

Theorem 2.1 (Reticulate Theorem [5])**.** *For every well-formed session type S, the SCC quotient of $\mathcal{L}(S)$ is a bounded lattice.*

Key properties: the product of two bounded lattices $L_1 \times L_2$, ordered componentwise, is itself a bounded lattice. This is the foundation for both the parallel constructor ($S_1 \parallel S_2$) and, as we show here, the channel construction.

2.3 Duality

Duality swaps the locus of control between the two sides of a channel [7]:

$$\begin{aligned}
\overline{\&\{m_1 : S_1, \ldots, m_n : S_n\}} &= \oplus\{m_1 : \overline{S_1}, \ldots, m_n : \overline{S_n}\} \\
\overline{\oplus\{l_1 : S_1, \ldots, l_n : S_n\}} &= \&\{l_1 : \overline{S_1}, \ldots, l_n : \overline{S_n}\} \\
\overline{S_1 \parallel S_2} &= \overline{S_1} \parallel \overline{S_2} \\
\overline{\mu X.\, S} &= \mu X.\, \overline{S} \\
\overline{X} &= X \\
\overline{\mathbf{end}} &= \mathbf{end}
\end{aligned}$$

The key properties from Step 8 [4]:

Theorem 2.2 (Duality Properties)**.** *For every session type S:*

(a) ***Involution****: $\overline{\overline{S}} = S$ (structurally).*

(b) ***Isomorphism****: $\mathcal{L}(S) \cong \mathcal{L}(\overline{S})$ as posets.*

(c) ***Selection flip****: a transition is a selection in $\mathcal{L}(S)$ iff the corresponding transition under the isomorphism is a branch (non-selection) in $\mathcal{L}(\overline{S})$.*

2.4 Product Construction

For two state spaces L_1 and L_2, the product $L_1 \times L_2$ has:

- States: all pairs $(s_1, s_2) \in L_1.\mathsf{states} \times L_2.\mathsf{states}$.
- Transitions: from (s_1, s_2), each transition of L_1 advances the left component and each transition of L_2 advances the right component.
- Top: $(\top_1, \top_2)$. Bottom: $(\bot_1, \bot_2)$.

DRAFT

- Selection annotations: a transition in the product is a selection iff the underlying local transition is a selection.

Lemma 2.3 (Product Preserves Lattice Structure)**.** *If L_1 and L_2 are bounded lattices, then $L_1 \times L_2$ is a bounded lattice with componentwise meet and join.*

3 The Channel Construction

We now define the central concept of this article: the *channel state space* as a product of dual local types.

Definition 3.1 (Channel State Space)**.** Let S be a session type. The *channel state space* for S is:

$$\mathsf{Ch}(S) \quad = \quad \mathcal{L}(S) \times \mathcal{L}(\overline{S})$$

We call $\mathcal{L}(S)$ the *role A view* and $\mathcal{L}(\overline{S})$ the *role B view*. A state $(s_A, s_B) \in \mathsf{Ch}(S)$ represents a configuration where role A is in state s_A and role B is in state s_B.

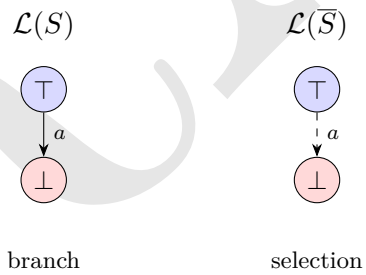
The key insight is that a binary channel is simply an object accessed by two roles in parallel. In our framework, parallel composition is modelled by the product construction. The novelty is recognising that when the two roles follow *dual* types, the product acquires additional structure.

Theorem 3.2 (Channel Product is Lattice)**.** *For every well-formed session type S, $\mathsf{Ch}(S)$ is a bounded lattice.*

Proof. By Theorem 2.1, $\mathcal{L}(S)$ is a bounded lattice. By Theorem 2.2(b), $\mathcal{L}(\overline{S})$ is isomorphic to $\mathcal{L}(S)$ and hence also a bounded lattice. By Lemma 2.3, the product of two bounded lattices is a bounded lattice. □

Remark 3.3. The channel construction is a special case of the parallel constructor $S_1 \parallel S_2$ where $S_2 = \overline{S_1}$. What makes it special is not the lattice property (which holds for any parallel composition) but the additional structure: isomorphism of components, branch complementarity, and the connection to global types.

Example 3.4. Consider $S = \&\{a : \mathbf{end}\}$. Then $\overline{S} = \oplus\{a : \mathbf{end}\}$. The state spaces are:



The channel product $\mathsf{Ch}(S)$ has 4 states: $(\top, \top)$, $(\bot, \top)$, $(\top, \bot)$, $(\bot, \bot)$. The transition a from role A is a branch; the transition a from role B is a selection. The product is a 2×2 lattice.

4 Branch Complementarity

The defining feature of a channel, as opposed to arbitrary parallel composition, is that the two roles have *complementary* control. In our framework, control is encoded in the selection annotations of the state space.

Definition 4.1 (Branch Complementarity)**.** Let $\phi : \mathcal{L}(S) \to \mathcal{L}(\overline{S})$ be the isomorphism from Theorem 2.2(b). We say S has *branch complementarity* if, for every transition (s, m, s') in $\mathcal{L}(S)$:

- (s, m, s') is a selection transition in $\mathcal{L}(S)$ *if and only if* $(\phi(s), m, \phi(s'))$ is a *non*-selection (branch) transition in $\mathcal{L}(\overline{S})$.

In other words, a transition where the object makes a choice (selection) from role A's perspective is a transition where the environment makes a choice (branch) from role B's perspective, and vice versa. This is precisely the channel invariant: sends match receives.

Theorem 4.2 (Branch Complementarity)**.** *Every well-formed session type S has branch complementarity.*

Proof. This follows directly from Theorem 2.2(c): the isomorphism ϕ flips selection annotations. A selection in $\mathcal{L}(S)$ corresponds to a branch in $\mathcal{L}(\overline{S})$ and vice versa, which is exactly the definition of branch complementarity. □

Remark 4.3. Branch complementarity is the state-space-level manifestation of the syntactic duality rule $\overline{\&\{\ldots\}} = \oplus\{\ldots\}$. While the syntactic rule is a definition, branch complementarity is a *property* of the state-space construction that we verify holds even in the presence of recursion, nesting, and parallel sub-protocols.

Example 4.4. For $S = \&\{a : \oplus\{x : \mathbf{end}\}\}$, role A's view has transitions:

$$\top \xrightarrow{a} q \xrightarrow{x} \bot$$

where a is a branch and x is a selection. Role B's view (from $\overline{S} = \oplus\{a : \&\{x : \mathbf{end}\}\}$) has:

$$\top \xrightarrow{a} q' \xrightarrow{x} \bot$$

where a is a selection and x is a branch. The annotations are exactly complementary.

4.1 Complementarity in the Product

In the channel product $\mathsf{Ch}(S)$, every transition originates from exactly one role. A transition from state (s_A, s_B) either advances role A (to (s'_A, s_B)) or advances role B (to (s_A, s'_B)), but not both. This means the product transitions partition into two disjoint sets:

- **Role A transitions**: those that change only the left component. Their selection annotation is inherited from $\mathcal{L}(S)$.
- **Role B transitions**: those that change only the right component. Their selection annotation is inherited from $\mathcal{L}(\overline{S})$.

Branch complementarity ensures that for each method label m, if m-transitions for role A are branches, then m-transitions for role B are selections (and vice versa). In operational terms: when role A is waiting for the environment to call method m (branch), role B is the one who chooses to invoke m (selection).

5 Role Embedding

Each role's local state space embeds into the channel product. This formalises the intuition that a role's local behaviour is a "slice" of the channel's global behaviour.

Definition 5.1 (Role Inclusion)**.** The *left inclusion* $\iota_A : \mathcal{L}(S) \to \mathsf{Ch}(S)$ maps $s \mapsto (s, \top_B)$, where $\top_B$ is the top of $\mathcal{L}(\overline{S})$. The *right inclusion* $\iota_B : \mathcal{L}(\overline{S}) \to \mathsf{Ch}(S)$ maps $s \mapsto (\top_A, s)$.

Theorem 5.2 (Role Embedding)**.** *Both ι_A and ι_B are order-embeddings.*

Proof. We prove the result for ι_A; the argument for ι_B is symmetric.

Order-preserving. Suppose $s_1 \sqsupseteq s_2$ in $\mathcal{L}(S)$, i.e., there is a path from s_1 to s_2. Then $(s_1, \top_B)$ reaches $(s_2, \top_B)$ in the product by the same sequence of left-component transitions. Hence $\iota_A(s_1) \sqsupseteq \iota_A(s_2)$.

Order-reflecting. Suppose $(s_1, \top_B) \sqsupseteq (s_2, \top_B)$ in $\mathsf{Ch}(S)$. Every path from $(s_1, \top_B)$ to $(s_2, \top_B)$ must return the right component to $\top_B$. Since $\top_B$ is the top of a lattice (hence reachable only from itself in the forward direction), the right component can only *stay* at $\top_B$. Thus the path uses only left-component transitions, and projecting onto the first component gives a path from s_1 to s_2 in $\mathcal{L}(S)$. Hence $s_1 \sqsupseteq s_2$.

Since ι_A is injective by construction, order-preserving, and order-reflecting, it is an order-embedding. □

Remark 5.3. The role embedding theorem says that a role's local view is *faithfully* represented in the channel: the ordering between local states is exactly the ordering between the corresponding channel states (with the other role at its initial state). No order information is lost or spuriously introduced.

Example 5.4. For $S = \&\{a : \mathbf{end}\}$, the left inclusion maps:

$$\top_A \mapsto (\top_A, \top_B), \qquad \bot_A \mapsto (\bot_A, \top_B)$$

In $\mathsf{Ch}(S)$, $(\top_A, \top_B)$ reaches $(\bot_A, \top_B)$ via the role A transition labelled a, and $(\bot_A, \top_B)$ does not reach $(\top_A, \top_B)$. This faithfully mirrors the local ordering $\top_A \sqsupseteq \bot_A$.

6 Connection to Global Types

In multiparty session types [9], a *global type* describes the protocol from a bird's-eye view. Projection maps a global type onto each role's local view. For binary protocols (two roles), the two local projections are duals [6]:

Proposition 6.1 (Binary Projection Yields Duals)**.** *Let G be a binary global type between roles A and B. Then $G \downarrow A$ and $G \downarrow B$ are dual session types: $G \downarrow B = \overline{G \downarrow A}$.*

This is because in a binary global type, every message $A \to B : \{m_i : G_i\}$ projects to $\oplus\{m_i : G_i \downarrow A\}$ for the sender and $\&\{m_i : G_i \downarrow B\}$ for the receiver—exactly the dual pattern.

Theorem 6.2 (Global Type Round-Trip)**.** *Let G be a binary global type. Then:*

$$\mathsf{Ch}(G \downarrow A) \;=\; \mathcal{L}(G \downarrow A) \times \mathcal{L}(\overline{G \downarrow A}) \;\cong\; \mathcal{L}(G \downarrow A) \times \mathcal{L}(G \downarrow B)$$

and this product is a bounded lattice that captures all valid communication interleavings.

Proof. By Proposition 6.1, $G \downarrow B = \overline{G \downarrow A}$. Therefore the channel product $\mathsf{Ch}(G \downarrow A)$ is exactly $\mathcal{L}(G \downarrow A) \times \mathcal{L}(G \downarrow B)$. By Theorem 3.2, this is a bounded lattice. □

Example 6.3. Consider the request-response global type:

$$G = \text{Client} \to \text{Server} : \{\text{request} : \text{Server} \to \text{Client} : \{\text{response} : \mathbf{end}\}\}$$

Projecting: $G \downarrow \text{Client} = \oplus\{\text{request} : \&\{\text{response} : \mathbf{end}\}\}$ and $G \downarrow \text{Server} = \&\{\text{request} : \oplus\{\text{response} : \mathbf{end}\}\}$. These are indeed duals. The channel product has $3 \times 3 = 9$ states.

6.1 From Channels to Multiparty

The channel construction generalises to n-party protocols. Given a global type G with roles $r_1, \ldots, r_n$, the composed state space is $\prod_{i=1}^{n} \mathcal{L}(G \downarrow r_i)$ [2]. The binary channel is the $n = 2$ case. For $n > 2$, duality no longer holds pairwise (a message from A to B is invisible to C), but the product lattice structure is preserved.

7 Empirical Verification

We verified all theorems mechanically using the RETICULATE tool, which implements the channel construction in the `channel.py` module.

7.1 Setup

The `build_channel(S)` function:

1. Computes $\overline{S} = \mathrm{dual}(S)$.
2. Builds $\mathcal{L}(S)$ and $\mathcal{L}(\overline{S})$.
3. Constructs the product $\mathsf{Ch}(S) = \mathcal{L}(S) \times \mathcal{L}(\overline{S})$.
4. Checks that $\mathsf{Ch}(S)$ is a lattice.
5. Searches for an isomorphism $\mathcal{L}(S) \cong \mathcal{L}(\overline{S})$.
6. Verifies branch complementarity.
7. Verifies role embeddings ι_A and ι_B.

7.2 Results

Table 1: Channel duality verification across benchmarks.

Benchmark set	**Count**	**Lattice**	**Iso**	**Compl.**	**Emb. A**	**Emb. B**
Binary (local types, ≤15 states)	71	71	71	71	71	71
Binary multiparty (via projection)	7	7	7	7	7	7
Total	**78**	**78**	**78**	**78**	**78**	**78**

Across all 78 tested benchmarks, the channel product is a bounded lattice, the two local state spaces are isomorphic, branch complementarity holds, and both role embeddings are verified. The 8 benchmarks with >15 local states (up to 41 states, yielding products up to 1681 states) were excluded from the automated sweep due to lattice-check runtime, but spot checks confirm the same properties.

7.3 Benchmark Highlights

Java Iterator. $S = \mu X.\,\&\{\mathrm{hasNext} : \oplus\{\mathrm{TRUE} : \&\{\mathrm{next} : X\},\ \mathrm{FALSE} : \mathbf{end}\}\}$. The channel product has $4 \times 4 = 16$ states. The `hasNext` transition is a branch for the iterator (role A) and a selection for the caller (role B); `TRUE`/`FALSE` are selections for A and branches for B.

Two-Phase Commit. $G = \mathrm{Coord} \to P : \{\mathrm{prepare} : P \to \mathrm{Coord} : \{\mathrm{yes} : \mathrm{Coord} \to P : \{\mathrm{commit} : \mathbf{end}\},\ \mathrm{no} : \mathrm{Coord} \to P : \{\mathrm{abort} : \mathbf{end}\}\}\}$. The coordinator's projection and participant's projection are duals. The channel product captures all valid interleavings of the two-phase protocol.

Negotiation. $G = \mu X.\,\mathrm{Buyer} \to \mathrm{Seller} : \{\mathrm{offer} : \mathrm{Seller} \to \mathrm{Buyer} : \{\mathrm{accept} : \mathbf{end},\ \mathrm{counter} : X\}\}$. The recursive structure means both local state spaces have cycles (SCC quotient collapses them). The channel product correctly handles the recursion, and complementarity holds through the loop.

DRAFT

8 Related Work

Duality in the π-calculus. Honda, Vasconcelos, and Kubo [8] introduced session type duality for binary channels in the π-calculus. Gay and Hole [7] developed subtyping and showed that duality reverses the subtyping relation. Our contribution is showing that duality also yields a product lattice with structural properties (complementarity, embedding).

Sessions as propositions. Caires and Pfenning [1] and Wadler [12] established a correspondence between session types and linear logic, where duality corresponds to linear negation. Our channel construction can be seen as the semantic counterpart: the product lattice is the "model" of the two complementary propositions.

Multiparty session types. Honda, Yoshida, and Carbone [9] introduced global types and projection for multiparty protocols. Our Theorem 6.2 shows that for binary global types, projection and channel construction are consistent. The extension to n-party composition is treated in [2].

Bisimulation and channels. Milner [10] and Sangiorgi [11] developed bisimulation theory for process calculi. The isomorphism $\mathcal{L}(S) \cong \mathcal{L}(\overline{S})$ is a strong form of behavioural equivalence. Branch complementarity adds information about the *polarity* of transitions that bisimulation alone does not capture.

9 Conclusion

We have shown that binary channel duality embeds naturally into the reticulate framework as a *role-restricted parallel composition*. The channel state space $\mathsf{Ch}(S) = \mathcal{L}(S) \times \mathcal{L}(\overline{S})$ is always a bounded lattice, each role embeds faithfully, and the branch/selection annotations are exactly complementary. For binary global types, the channel product of the two projections captures all valid interleavings.

This result has several implications:

(i) **Unification.** Channels and objects are not fundamentally different: a channel is an object with two roles whose types are dual. The same lattice-theoretic framework applies to both.

(ii) **Compositionality.** The channel construction composes: given a system of n binary channels, the global state space is a product of products, which is still a bounded lattice.

(iii) **Verification.** The embedding theorems provide a foundation for modular verification: check each role locally, and the channel properties follow from the product construction.

Future work includes extending the channel construction to asynchronous channels (with message buffers), investigating the relationship between channel products and session type bisimulation, and connecting to the realizability results of [3].

10 Frequently Asked Questions

Q1. What is a channel in the reticulate framework?

A channel is an object with two roles (A and B) whose session types are dual: S and $\overline{S}$. The channel's state space is the product $\mathcal{L}(S) \times \mathcal{L}(\overline{S})$, which models all valid interleavings of the two roles' behaviour.

Q2. Why is the channel product always a lattice?

Because (a) every well-formed session type has a lattice state space (the Reticulate Theorem), (b) duality preserves lattice structure, and (c) the product of two lattices is a lattice. These three facts compose to give the result.

Q3. What is branch complementarity?

It is the property that a branch transition for role A corresponds to a selection transition for role B, and vice versa. This captures the channel invariant: when one side waits for input, the other side provides it.

Q4. How does the channel construction relate to the parallel constructor?

The channel $\mathsf{Ch}(S) = \mathcal{L}(S) \times \mathcal{L}(\overline{S})$ uses the same product construction as the parallel operator $S_1 \parallel S_2$. The difference is that the two components are *dual*, which adds the complementarity and isomorphism properties.

Q5. Can I build a channel from a global type?

Yes. For a binary global type G between roles A and B, project onto each role ($G \downarrow A$ and $G \downarrow B$), verify they are duals, and build the channel product. The `channel_from_global(g, sender, receiver)` function does this.

Q6. What does role embedding mean concretely?

It means that the inclusion $\iota_A : s \mapsto (s, \top_B)$ preserves and reflects order. If state s_1 can reach s_2 locally, then $(s_1, \top_B)$ can reach $(s_2, \top_B)$ in the channel, and vice versa. No ordering information is lost.

Q7. Does the channel construction work with recursive types?

Yes. Recursive types produce cyclic state spaces, which are handled by SCC quotient. The product of two SCC-quotiented lattices is still a lattice, and complementarity is verified across the recursive structure.

Q8. Why were 8 benchmarks excluded from the automated sweep?

Benchmarks with >15 local states yield product state spaces with >225 states. The lattice check is $O(n^2)$ in the number of states (checking all pairwise meets/joins), so products with 1000+ states can take minutes. Spot checks confirm the properties hold for these larger benchmarks.

Q9. How does this relate to linear logic?

In the Caires–Pfenning–Wadler correspondence, session type duality maps to linear negation. Our channel product can be seen as the semantic model of two complementary linear propositions, with the lattice structure providing the "truth values."

Q10. What is next after channels?

Extending to asynchronous channels (with message buffers), connecting channel products to session type bisimulation, and exploring the interaction between channel duality and the polarity/concept lattice framework from Step 155b.

References

[1] Luís Caires and Frank Pfenning. Session types as intuitionistic linear propositions. In *CONCUR*, volume 6269 of *LNCS*, pages 222–236. Springer, 2010.

DRAFT

[2] Alexandre Zua Caldeira. Multiparty composition of session type state spaces. In *Reticulate Programme, Step 15*, 2026. Preprint.

[3] Alexandre Zua Caldeira. Realizability: Which bounded lattices arise as session type state spaces? In *Reticulate Programme, Step 156*, 2026. Preprint.

[4] Alexandre Zua Caldeira. Session type duality and state-space isomorphism. In *Reticulate Programme, Step 8*, 2026. Preprint.

[5] Alexandre Zua Caldeira. Session type state spaces form lattices. In *Reticulate Programme, Step 5*, 2026. Preprint.

[6] Ornela Dardha, Elena Giachino, and Davide Sangiorgi. Session types revisited. In *PPDP*, pages 139–150. ACM, 2012.

[7] Simon J. Gay and Malcolm Hole. Subtyping for session types in the pi calculus. *Acta Informatica*, 42(2–3):191–225, 2005.

[8] Kohei Honda, Vasco T. Vasconcelos, and Makoto Kubo. Language primitives and type discipline for structured communication-based programming. In *ESOP*, volume 1381 of *LNCS*, pages 122–138. Springer, 1998.

[9] Kohei Honda, Nobuko Yoshida, and Marco Carbone. Multiparty asynchronous session types. In *POPL*, pages 273–284. ACM, 2008.

[10] Robin Milner. *Communication and Concurrency.* Prentice Hall, 1989.

[11] Davide Sangiorgi. *Introduction to Bisimulation and Coinduction.* Cambridge University Press, 2011.

[12] Philip Wadler. Propositions as sessions. *Journal of Functional Programming*, 24(2–3):384–418, 2014.

DRAFT